



Bilkent University

EEE 443 - Neural Networks

Fourth Assignment Report

Fall 2025–2026

Aleyna Yaren Öztürken - 22103420

December 1, 2025

Contents

1	Introduction	1
2	Methodology	1
2.1	Data Preprocessing	1
2.2	Network Architecture	1
2.3	Activation Functions	2
2.4	Backpropagation Algorithm	2
2.5	Cost Functions	3
2.5.1	Online Learning	3
2.5.2	Batch Learning	4
3	Experimental Setup	5
4	Results and Discussion	6
4.1	Analysis of Case 1 Online Learning(Sigmoid with Thresholds)	6
4.2	Analysis of Case 2 Online Learning(Sigmoid with No Thresholds)	9
4.3	Analysis of Case 3 Online Learning(Softmax with Thresholds)	12
4.4	Analysis of Case 4 Online Learning(Softmax with No Thresholds)	15
4.5	Overall Comparison of Online Learning Cases	17
4.6	Analysis of Batch Learning with Case 1 Configuration Batch Size = 10	18
4.7	Analysis of Batch Learning with Case 1 Configuration Batch Size = 100	21
5	Overall Comments for All Experiments	23
6	Conclusion	24

1 Introduction

In this assignment a 5-layer Convolutional Neural Network (CNN) for handwritten digit recognition had been implemented and analysed using the MNIST dataset. The objective of this assignment is to understand the mechanics of CNNs and the Backpropagation algorithm. The network architecture is a simplified version of LeNet-5, implemented from scratch using Python and NumPy without using any special machine learning libraries. The task involves classifying 28×28 grayscale images of digits (0-9). The specific constraints include using linear activation for hidden layers, shared weights for convolutional kernels, and manual derivation of gradients. Investigation of four different configurations involving activation functions (Sigmoid vs. Softmax) and bias thresholds (with threshold vs without threshold) had been done. Furthermore, the impact of batch learning versus online learning are investigated detailly.

2 Methodology

The methodology that followed in this assignment focuses on low-level implementation of the CNN training pipeline, ensuring all operations satisfy to the assignment's constraints without reliance on automatic differentiation frameworks. The process begins with data preprocessing and integer labels are converted into one-hot encoded vectors. Parameter initialization is performed using a Gaussian distribution ($\mu = 0, \sigma = 0.1$) to break symmetry and facilitate effective start-up learning. The core training algorithm utilizes Stochastic Gradient Descent (SGD); in each iteration, the network performs a forward pass to compute activations, calculates the error based on the selected cost function (Mean Squared Error or Cross-Entropy), and executes a manual backward pass to derive gradients via the chain rule. A critical aspect of our methodology involves the custom handling of weight updates for Layer H3, where gradients are explicitly accumulated across the shared filter connections to satisfy the specific architectural design before applying the weight updates.

2.1 Data Preprocessing

The MNIST dataset consists of 60,000 training images and 10,000 test images. The raw data is stored in a binary format which was parsed using Python's gzip and struct modules. Before feeding the data into the network, two critical preprocessing steps were applied. First, the pixel intensity values, originally integers in the range $[0, 255]$, were normalized to floating-point values in the range $[0.0, 1.0]$. This normalization is essential for preventing saturation in the activation functions and ensuring stable gradient descent. Second, the integer labels were converted into One-Hot Encoded vectors (e.g., the digit 2 becomes $[0, 0, 1, 0, \dots]$). This format aligns with the output layer of the network, allowing for the calculation of error vectors during backpropagation.

2.2 Network Architecture

The implemented network follows a hierarchical structure designed to extract the patterns of inputs. The architecture is defined as follows:

1. **Layer H1 (Convolution):** The input 28×28 image is convolved with 4 distinct filters of size 5×5 . Using valid convolution with no padding, results in 4 feature maps of size 24×24 . This layer is responsible for detecting low-level features such as edges and corners.
2. **Layer H2 (Average Pooling):** This layer performs downsampling to reduce the spatial dimensionality and sensitivity to shifts. A fixed 2×2 window with a stride of 2 computes the average of non-overlapping regions, reducing the 4 maps to 12×12 . The weights in this layer are fixed to $1/4$ for each pixel in the window and they are not updated during training.

3. **Layer H3 (Convolution - Shared Weights):** This layer implements a specific connectivity pattern to reduce parameters. It contains only 3 distinct 5×5 filter kernels. However, each of these 3 kernels is applied to each of the 4 output maps from layer H2 independently. This creates $4 \times 3 = 12$ output feature maps of size 8×8 . During backpropagation, the gradients for a shared kernel are calculated by summing the contributions from all maps it processed.
4. **Layer H4 (Average Pooling):** Similar to H2, this layer performs 2×2 average pooling on the 12 feature maps from H3, resulting in 12 maps of size 4×4 .
5. **Layer H5 (Fully Connected):** The 12 feature maps are flattened into a single vector of length 192 ($12 \times 4 \times 4$). This vector is fully connected to 10 output neurons. The final activation is configurable between Sigmoid and Softmax.

2.3 Activation Functions

The activation function is a critical component that determines the output of a neuron given an input or set of inputs. In this specific architecture, the hidden layers (H1 through H4) utilize a linear activation function, effectively passing the weighted sum directly to the next layer. Non-linearity is introduced primarily at the output layer (H5). Two distinct configurations were implemented for the output neurons:

1. **Sigmoid Activation (Case 1 & 2):** The logistic sigmoid function maps the input to a probability-like value between 0 and 1. It is differentiable and monotonic.

$$\sigma(v) = \frac{1}{1 + e^{-v}}$$

When using Sigmoid, the cost function employed is the Mean Squared Error (MSE).

2. **Softmax Activation (Case 3 & 4):** The Softmax function is standard for multi-class classification problems. It normalizes the output vector such that the sum of all components equals 1, representing a probability distribution over the 10 digit classes.

$$\text{Softmax}(v_i) = \frac{e^{v_i}}{\sum_j e^{v_j}}$$

When using Softmax, the cost function employed is Categorical Cross-Entropy.

2.4 Backpropagation Algorithm

The training process relies on the Backpropagation algorithm to minimize the cost function. Unlike standard Multi-Layer Perceptrons (MLPs), applying backpropagation to this CNN requires handling spatial dimensions and parameter sharing. The process involves calculating the gradient of the loss function with respect to each filter weight and bias, then updating them via Gradient Descent.

The procedure is divided into three phases:

1. **Forward Pass:** The input image is convolved with filters (H1), downsampled (H2), convolved again with shared weights (H3), and downsampled (H4) before being flattened for the final fully connected layer (H5).
2. **Backward Pass:** The error is propagated from the output layer back to the input.
 - For the Fully Connected Layer (H5), gradients are computed via standard matrix multiplication.

- For Pooling Layers (H4, H2), the error is "upsampled." Since average pooling was used during the forward pass, the error gradient is distributed uniformly to the corresponding 2×2 receptive field in the previous layer.
 - For Convolutional Layers (H3, H1), the gradient calculation involves a "backward convolution" (correlation) between the error maps and the rotated kernels. Crucially, for Layer H3, which uses Shared Weights, the gradient for a specific kernel is the sum of gradients computed from all feature maps where that kernel was applied.
3. **Parameter Update:** The calculated gradients (∇W) are subtracted from the current weights, scaled by the learning rate η :

$$W_{new} = W_{old} - \eta \cdot \nabla W$$

2.5 Cost Functions

The cost function quantifies the discrepancy between the network's predicted output and the actual target label. Minimizing this value is the primary objective of the training process. In this assignment, the choice of cost function is strictly coupled with the choice of the output activation function to ensure mathematical compatibility and efficient gradient calculation.

1. **Mean Squared Error (MSE):** This cost function was utilized in conjunction with the Sigmoid activation (Cases 1 & 2). It calculates the average of the squared differences between the desired output (d_k) and the actual network output (y_k) over all output neurons. A scaling factor of $\frac{1}{2}$ is included to simplify the derivative calculation during backpropagation.

$$E = \frac{1}{2} \sum (d_k - y_k)^2$$

While often associated with regression problems, MSE is effectively used here for classification by treating the output vector as a Euclidean coordinate to be matched to the target one-hot vector. The gradient of this loss with respect to the weights incorporates the derivative of the sigmoid function, $\sigma'(v) = \sigma(v)(1 - \sigma(v))$.

2. **Categorical Cross-Entropy:** This function was paired with the Softmax activation (Cases 3 & 4). Rooted in information theory, it measures the dissimilarity between two probability distributions: the true distribution (the one-hot encoded target) and the predicted distribution (the Softmax output).

$$E = - \sum d_k \log(y_k)$$

Since the target vector d is one-hot encoded (containing a single '1' and nine '0's), this formula simplifies to the negative log probability of the correct class. A small epsilon value (10^{-9}) is typically added to the logarithm during implementation to prevent numerical instability ($\log(0)$). The combination of Softmax and Cross-Entropy typically results in a simplified gradient term $(y_k - d_k)$, which often accelerates learning in standard classification tasks.

2.5.1 Online Learning

Online Learning involves updating the network parameters immediately after processing a single training image. This approach introduces stochastic noise into the gradient estimation, as the direction is determined by a single example rather than the entire dataset. This noise can be advantageous, helping the network escape shallow local minima and often resulting in faster convergence per epoch in terms of error reduction, as the weights are updated 60,000 times in a single pass through the data.

However, it prevents the use of vectorized batch operations, making the computation time per epoch higher compared to batched approaches.

Algorithm 1: Online Learning Procedure (CNN)

Input: Training set $\mathcal{S} = \{(X^{(i)}, y^{(i)})\}_{i=1}^N$. Learning rate $\eta = 0.01$. Epochs.

Output: Trained Filters K_{H1} , K_{H3} and Weights W_{H5} .

- **Initialize:** Filters and weights with random values (Normal distribution, $\mu = 0, \sigma = 0.1$). Initialize thresholds if enabled.
 - For each epoch:
 - **Shuffle:** Randomly permute the training data.
 - For each image $X^{(i)}$ and label $y^{(i)}$:
 - * **1. Forward Propagation:**
 - $H1 \leftarrow$ Convolve $X^{(i)}$ with 4 filters.
 - $H2 \leftarrow$ Average Pool H1 (2×2).
 - $H3 \leftarrow$ Convolve H2 with 3 shared filters.
 - $H4 \leftarrow$ Average Pool H3 (2×2).
 - $Z \leftarrow$ Flatten H4 and compute dot product with W_{H5} .
 - $Output \leftarrow$ Apply Activation (Sigmoid/Softmax).
 - * **2. Backward Propagation:**
 - Compute Loss Gradient δ_{out} at Output.
 - Backpropagate δ through H5 to H4.
 - Upsample error through Pooling ($H4 \rightarrow H3$).
 - Backpropagate through Convolution (H3), accumulating gradients for shared filters.
 - Upsample error through Pooling ($H2 \rightarrow H1$).
 - Backpropagate through Convolution (H1).
 - * **3. Weight Update:**
 - Update all filters and weights: $\theta \leftarrow \theta - \eta \nabla \theta$.
-

2.5.2 Batch Learning

Batch Learning processes a specific subset of B training examples before performing a single weight update. The gradients are accumulated for all examples in the batch, and the average gradient is used for the update step. By averaging the noise over B samples, batch learning provides a more stable estimate of the true gradient direction. Computationally, it allows for matrix-matrix operations (if fully vectorized), which is significantly faster than processing images one by one. However, with large batch sizes, the number of updates per epoch drops significantly (e.g., $B = 100$ means only 600 updates per epoch vs 60,000 in online learning), which often leads to slower convergence in terms of epochs required to reach a specific accuracy.

Algorithm 2: Batch Learning Procedure

Input: Training set $\mathcal{S}$. Batch Size $B \in \{10, 100\}$.

Output: Trained Network Parameters.

- **Initialize:** Weights and Filters.
 - For each epoch:
 - **Reset Accumulators:** $\Delta W \leftarrow 0, \Delta K \leftarrow 0$.
 - For each batch of B images:
 - * For each image in batch:
 - Perform **Forward Pass**.
 - Perform **Backward Pass**.
 - **Accumulate Gradients:** $\Delta\theta_{total} \leftarrow \Delta\theta_{total} + \nabla\theta_{sample}$.
 - * **Update Weights (Once per Batch):**
 - Average the gradient: $\nabla_{avg} \leftarrow \frac{1}{B}\Delta\theta_{total}$.
 - Update parameters: $\theta \leftarrow \theta - \eta\nabla_{avg}$.
 - Reset Accumulators to zero.
-

3 Experimental Setup

The experiments were implemented using Python and NumPy. The code was optimized to run on a local CPU environment. To manage the computational load of convolution in Python, vectorization technique was utilized.

The experimental process consisted of two stages:

1. **Online Learning:** Four specific cases were tested with online learning to determine the optimal activation and threshold configuration.
 - **Case 1:** Sigmoid Activation, With Thresholds.
 - **Case 2:** Sigmoid Activation, No Thresholds.
 - **Case 3:** Softmax Activation, With Thresholds.
 - **Case 4:** Softmax Activation, No Thresholds.
2. **Batch Learning:** The best configuration from Online Learning was selected, and the effect of Batch Size ($B = 10, 100$) was analyzed.

Fixed Hyperparameters:

- **Learning Rate (η):** 0.01 (Fixed for all runs).
- **Filter Size:** 5×5 .
- **Epochs:** 30 or 50 (depending on convergence).

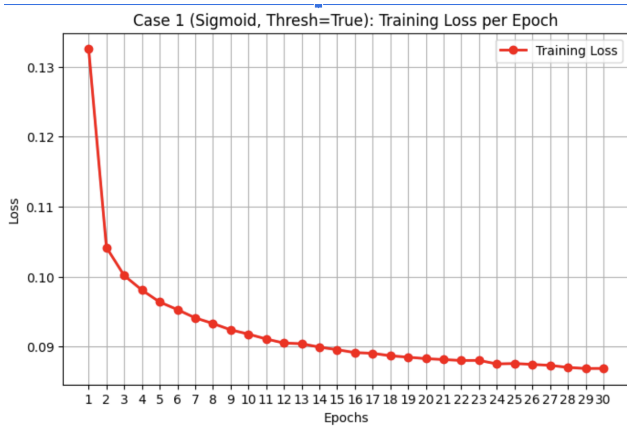
4 Results and Discussion

4.1 Analysis of Case 1 Online Learning(Sigmoid with Thresholds)

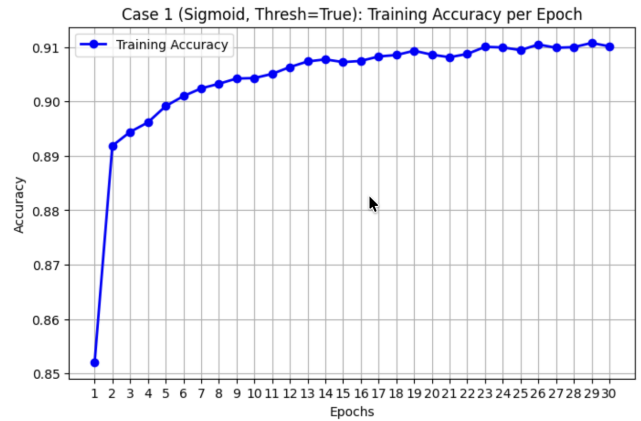
Case 1 proved to be the most stable and accurate configuration for this specific architecture.

Metric	Value
<i>Configuration Parameters</i>	
Epochs	30
Activation	Sigmoid
Thresholds	Yes
Batch Size	1 (Online)
<i>Performance</i>	
Total Training Duration	4760.30 s
Final Training Error	0.0868
Final Training Accuracy	91.01%
Final Test Accuracy	91.67%
Final Test Error	0.0839
Avg. Inference Time	0.9190 ms / pattern

Table 1: Performance Metrics for Case 1 Sigmoid with Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 1: Training performance for Case 1 (Sigmoid, Thresh=True) showing Loss (left) and Accuracy (right).

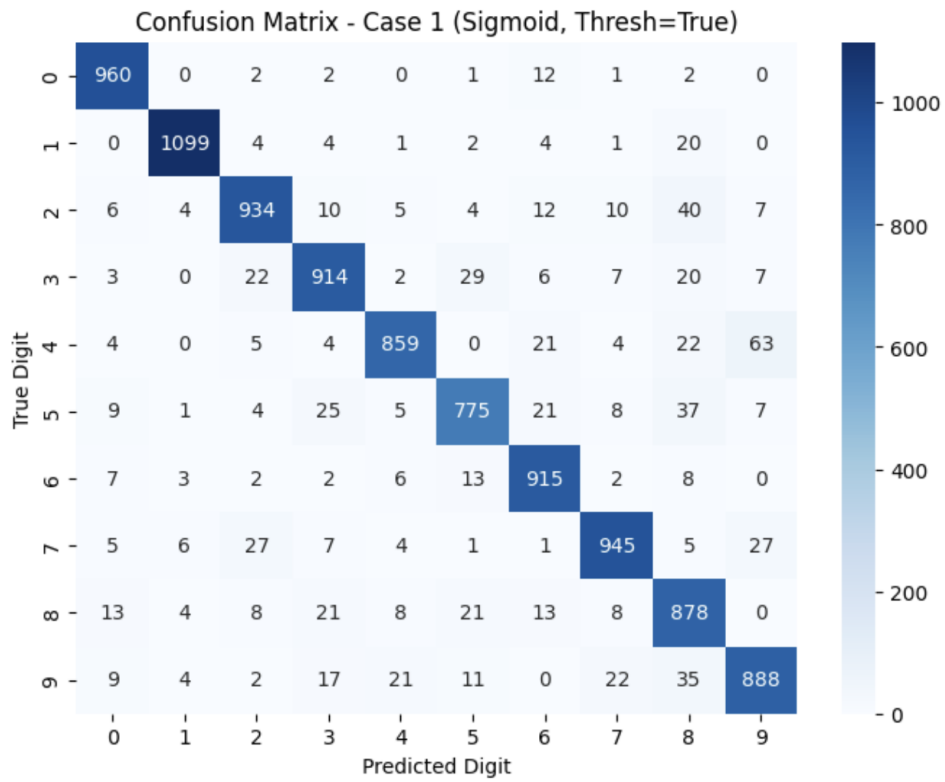
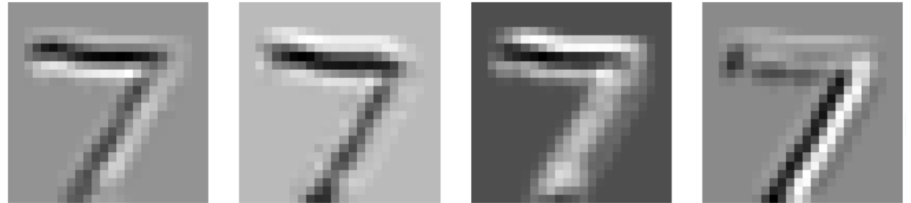


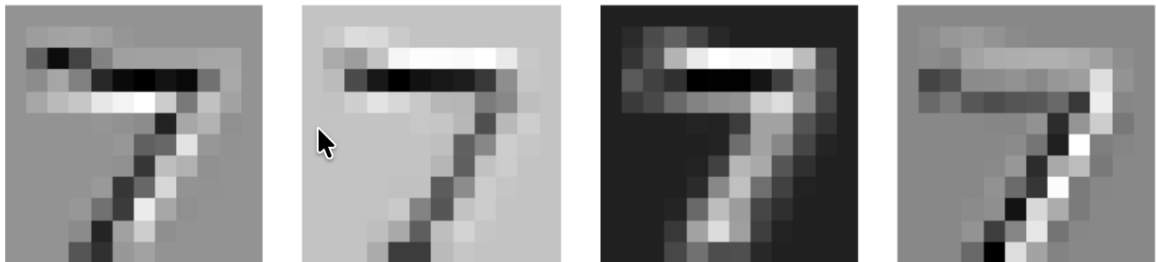
Figure 2: Confusion Matrix for Case 1 (Sigmoid, Thresh=True) on Test Set.



(a) Input: 7

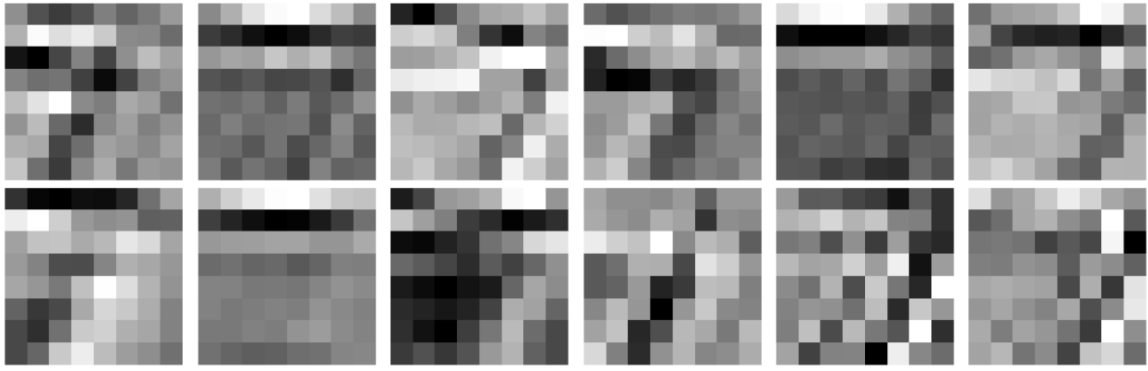


(b) Layer H1 (Conv 5x5)

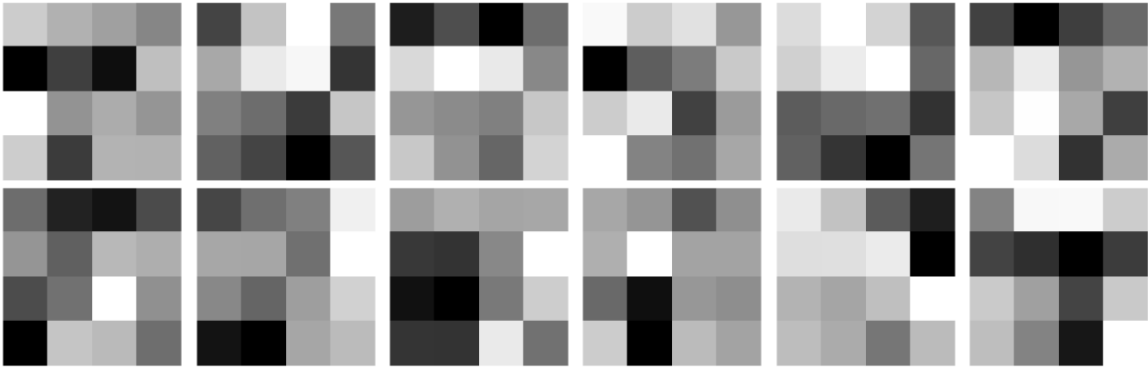


(c) Layer H2 (Pool 2x2)

Figure 3: Convolutional Layers (Part 1): Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

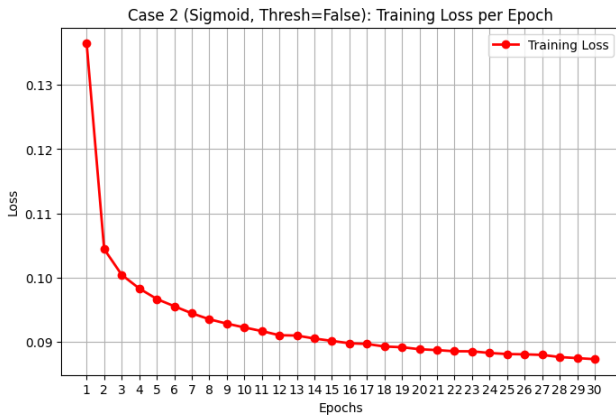
Figure 3: Convolutional Layers: Deeper layers.

The results demonstrate a functional Convolutional Neural Network trained with Sigmoid activation, achieving a respectable final test accuracy of 91.21% with stable convergence shown in the learning curves. The model learns effectively, as evidenced by the sharp drop in loss during the first five epochs and the layers (H1 through H4) clearly retaining the structural integrity of the input "7" through the convolutional and pooling stages. However, the training efficiency is a significant bottleneck; using online learning caused the process to take nearly 75 minutes (4469s) for just 30 epochs. However, the confusion matrix demonstrates highly accurate predictions.

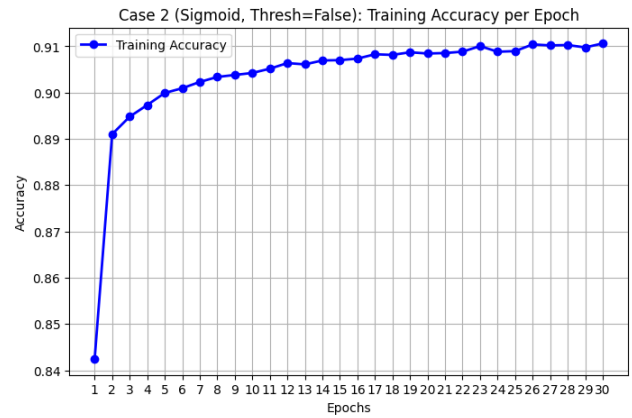
4.2 Analysis of Case 2 Online Learning(Sigmoid with No Thresholds)

Metric	Value
<i>Configuration Parameters</i>	
Epochs	30
Activation	Sigmoid
Thresholds	No
Batch Size	1 (Online)
<i>Performance</i>	
Total Training Duration	4469.19 s
Final Training Error	0.0872
Final Training Accuracy	91.06%
Final Test Accuracy	91.21%
Final Test Error	0.0844
Avg. Inference Time	0.9329 ms / pattern

Table 2: Performance Metrics for Case 2 Sigmoid No Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 4: Training performance for Case 2 (Sigmoid, Thresh=False) showing Loss (left) and Accuracy (right).

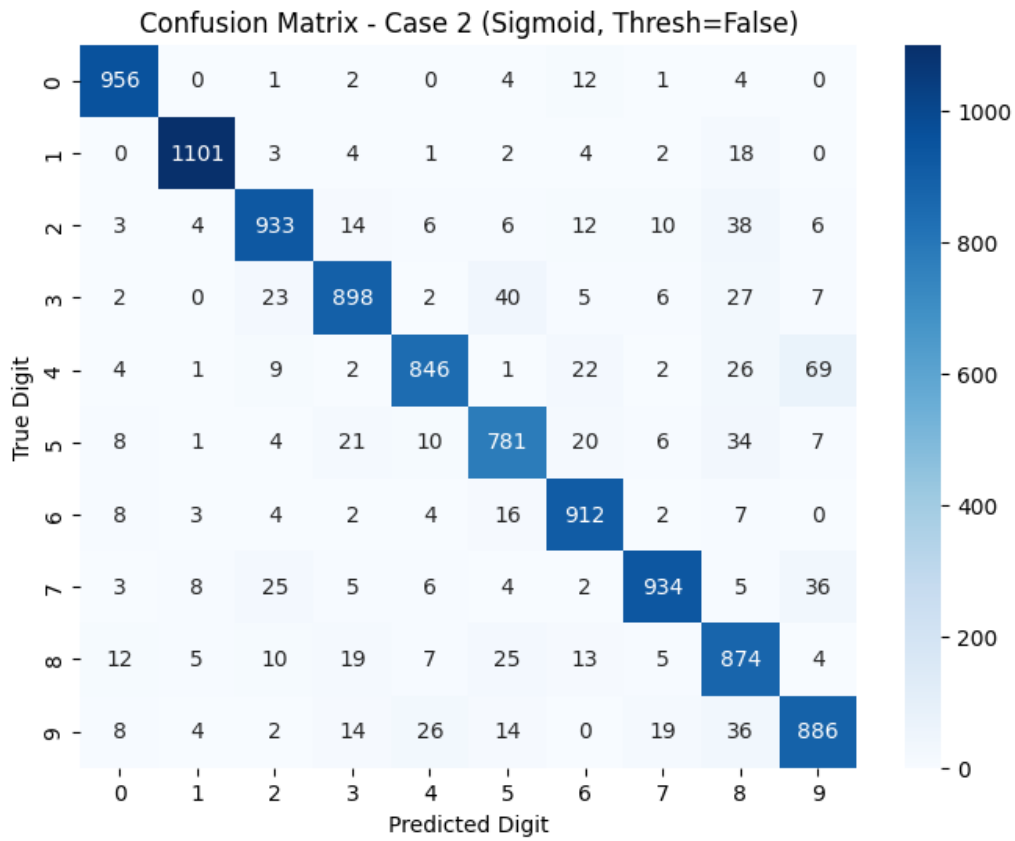


Figure 5: Confusion Matrix for Case 2 (Sigmoid, Thresh=False) on Test Set.

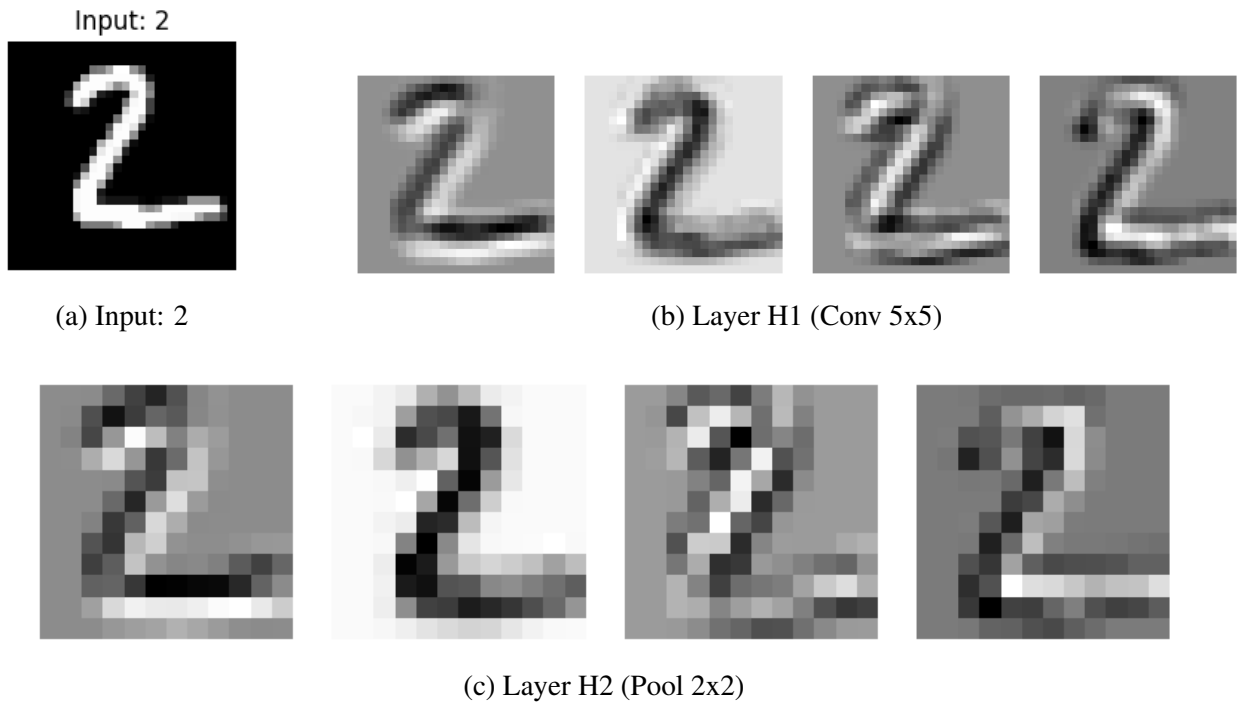
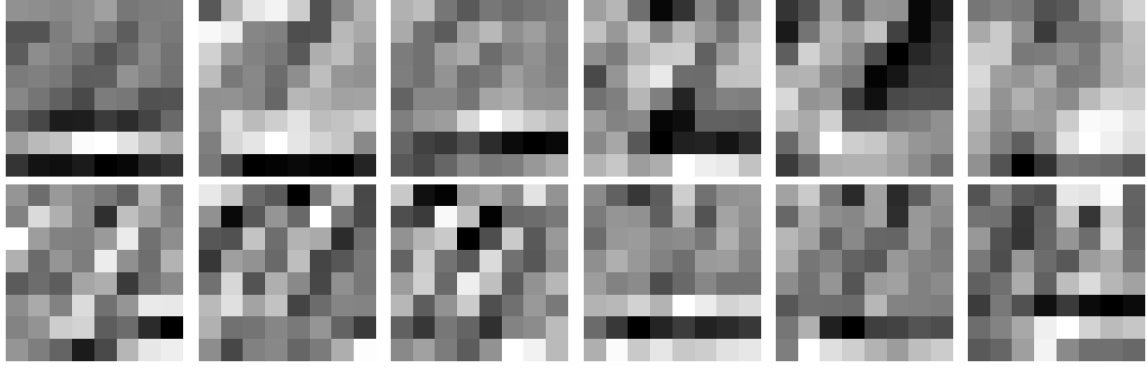
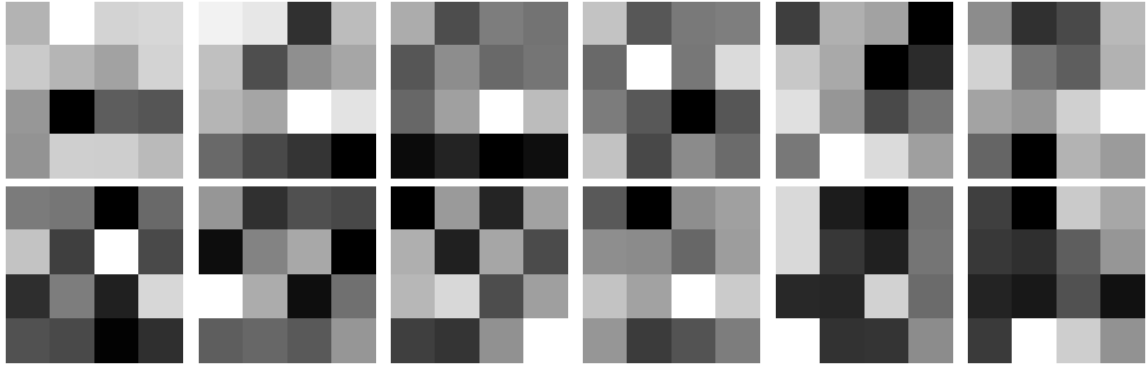


Figure 6: Convolutional Layers: Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

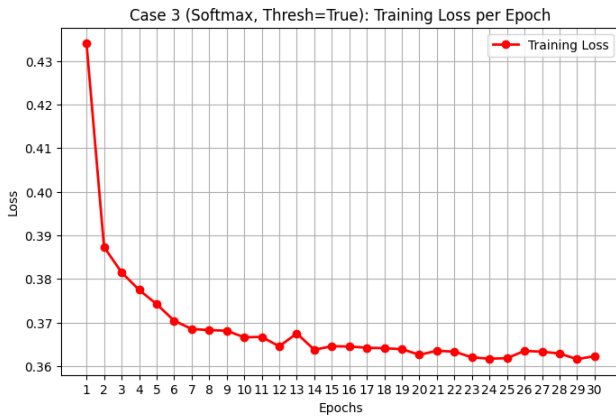
Figure 6: Convolutional Layers: Deeper layers.

The experimental results for Case 2 demonstrate a robust classification performance, achieving a final test accuracy of 91.21%. Notably, this metric slightly exceeds the training accuracy of 91.06%, indicating that the model generalizes well to unseen data without overfitting. While the use of online learning resulted in a substantial total training duration of 4469.19 seconds, the convergence behavior illustrated in Figure 4 remains smooth and monotonic, reflecting a stable optimization process absent of the volatility often seen in threshold-based approaches. The confusion matrix in Figure 5 highlights that the majority of classification errors arise from topologically similar digits, specifically between the pairs (4, 9) and (2, 7). Furthermore, the internal visualizations in Figure 6 confirm the network's hierarchical learning capability, transitioning from clear edge detection in the early convolutional layers (H1, H2) to increasingly abstract feature representations in the deeper pooling layers (H3, H4).

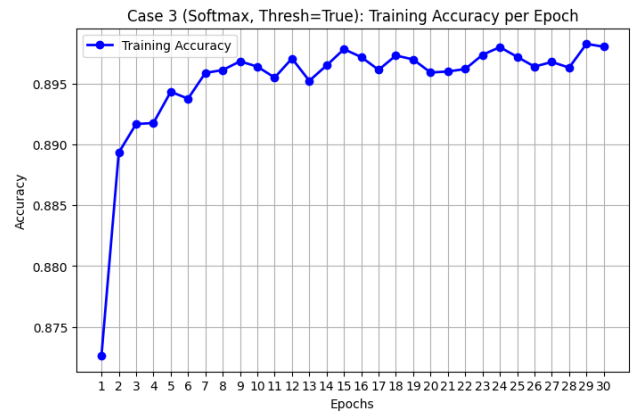
4.3 Analysis of Case 3 Online Learning(Softmax with Thresholds)

Metric	Value
<i>Configuration Parameters</i>	
Epochs	30
Activation	Softmax
Thresholds	Yes
Batch Size	1 (Online)
<i>Performance</i>	
Total Training Duration	5351.96 s
Final Training Error	0.3623
Final Training Accuracy	89.80%
Final Test Accuracy	90.66%
Final Test Error	0.3327
Avg. Inference Time	0.8698 ms / pattern

Table 3: Performance Metrics for Case 3 Softmax with Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 7: Training performance for Case 3 (Softmax, Thresh=True) showing Loss (left) and Accuracy (right).

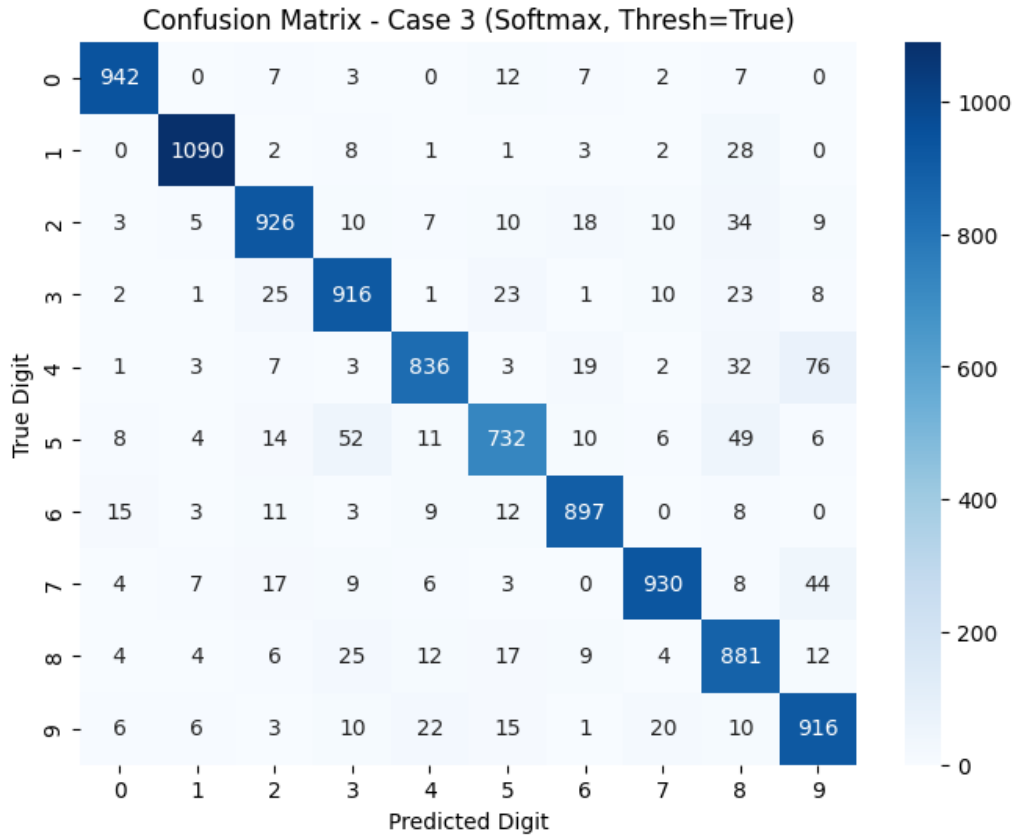


Figure 8: Confusion Matrix for Case 3 (Softmax, Thresh=True) on Test Set.

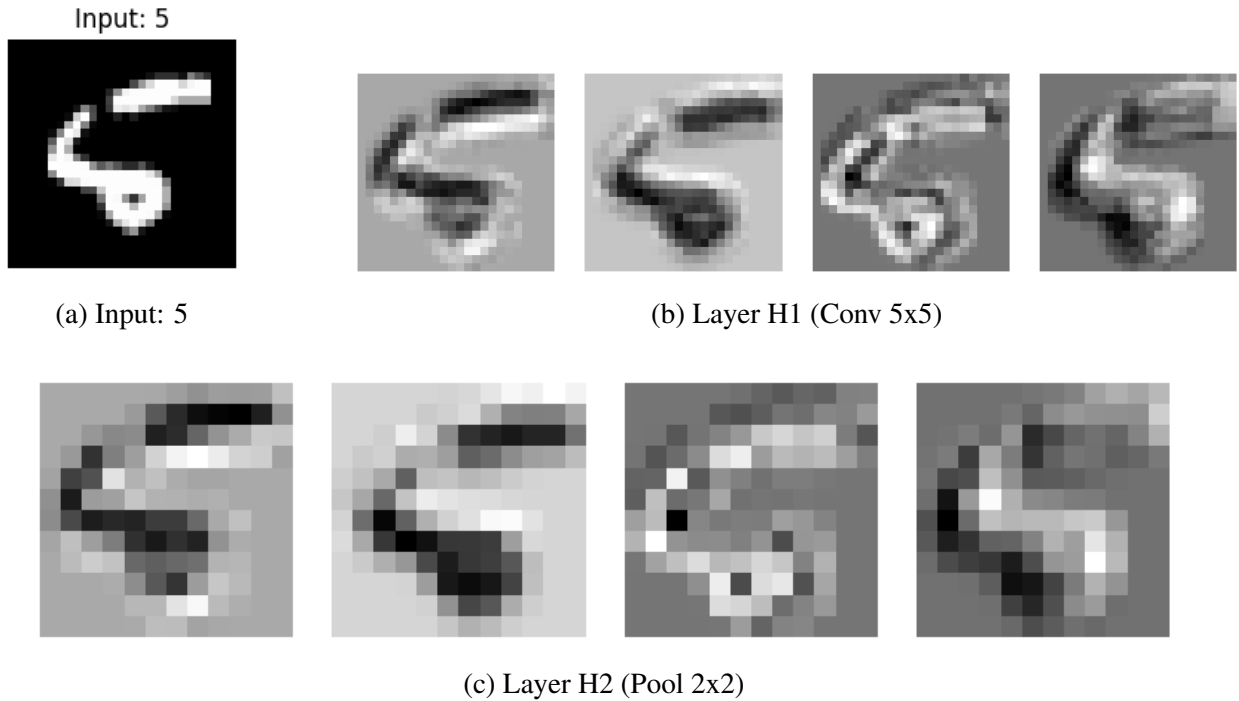
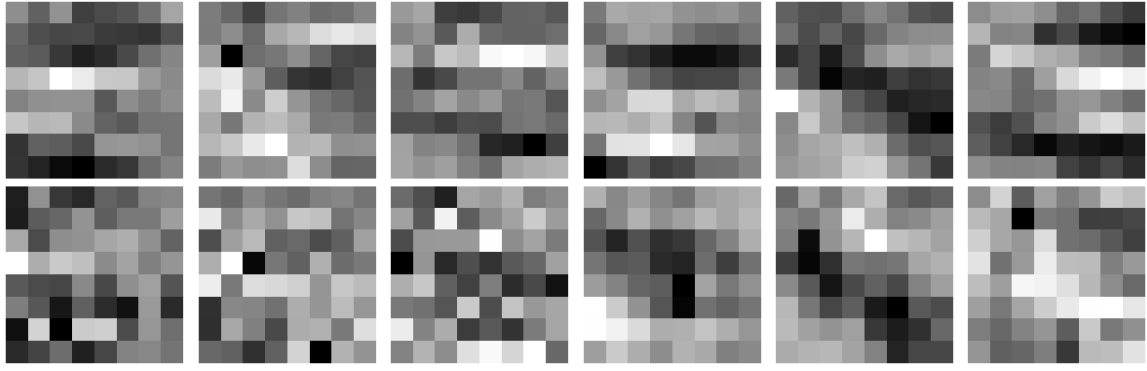
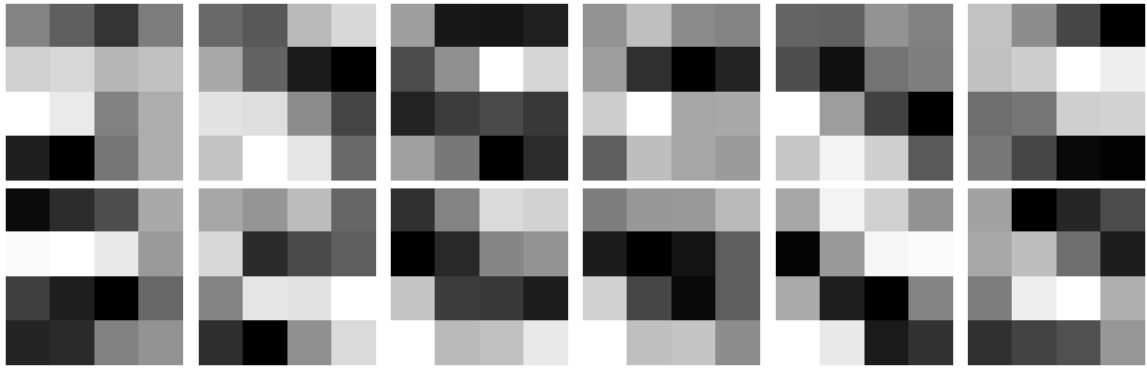


Figure 9: Convolutional Layers: Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

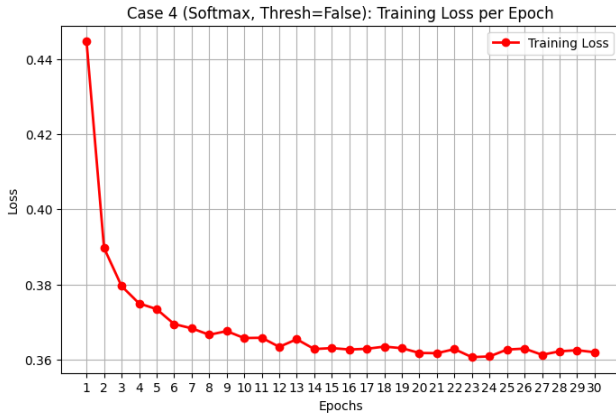
Figure 9: Convolutional Layers: Deeper layers.

The results for Case 3 indicate that while the model achieves a respectable Final Test Accuracy of 90.66%, the introduction of thresholds impacts the stability and cost of training. As detailed in Table 3, the total training duration increased to 5351.96 seconds with a higher Final Training Error of 0.3623 compared to previous cases. Figure 7 illustrates this volatility. Instead of the smooth convergence seen in previous tests, the accuracy curve bounces up and down. This indicates that the hard rules of thresholding introduce instability while the model is trying to minimize error. The Confusion Matrix (Figure 8) validates the accuracy metrics but highlights specific topological weaknesses: the model struggles significantly with the digit '4', misclassifying it as '9' in 76 instances, and also confuses '5' with '3' (52 instances). Despite these challenges, the visualisations in Figure 9 confirm that the convolutional layers effectively extract hierarchical features, transitioning from raw edge detection in the input '5' to abstract representations in deeper layers.

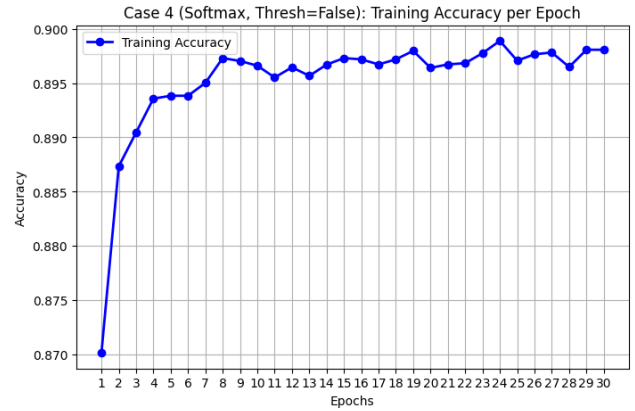
4.4 Analysis of Case 4 Online Learning(Softmax with No Thresholds)

Metric	Value
<i>Configuration Parameters</i>	
Epochs	30
Activation	Softmax
Thresholds	No
Batch Size	1 (Online)
<i>Performance</i>	
Total Training Duration	4419.45 s
Final Training Error	0.3619
Final Training Accuracy	89.81%
Final Test Accuracy	90.62%
Final Test Error	0.3328
Avg. Inference Time	0.8976 ms / pattern

Table 4: Performance Metrics for Case 4 Softmax with No Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 10: Training performance for Case 4 (Softmax, Thresh=False) showing Loss (left) and Accuracy (right).

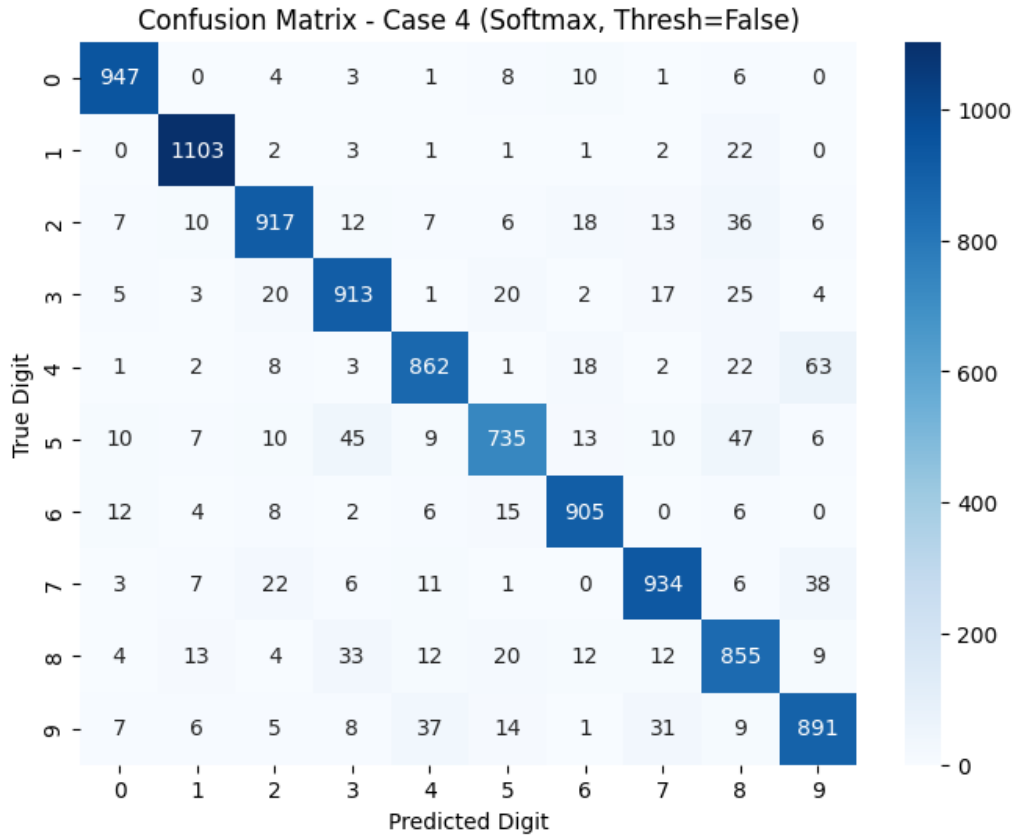


Figure 11: Confusion Matrix for Case 4 (Softmax, Thresh=False) on Test Set.

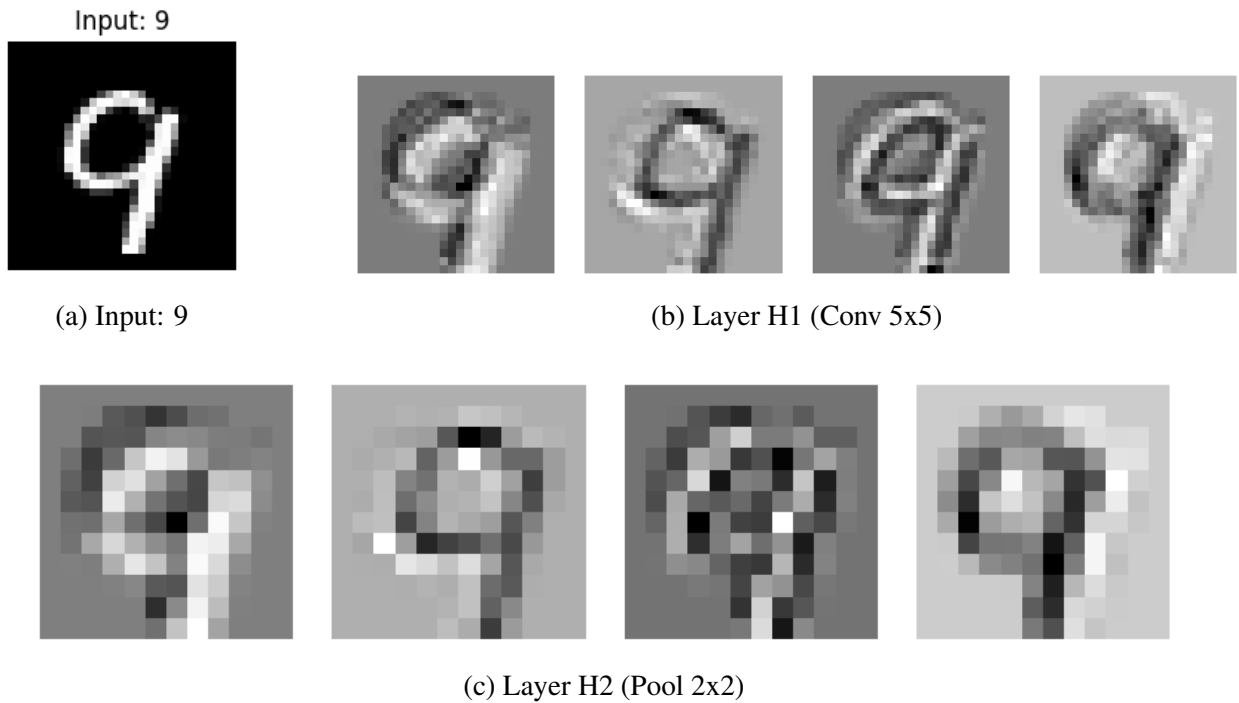
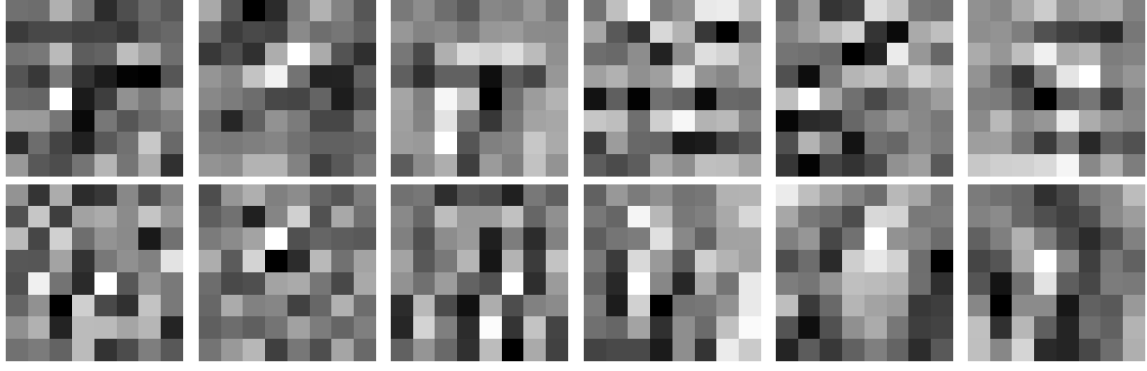
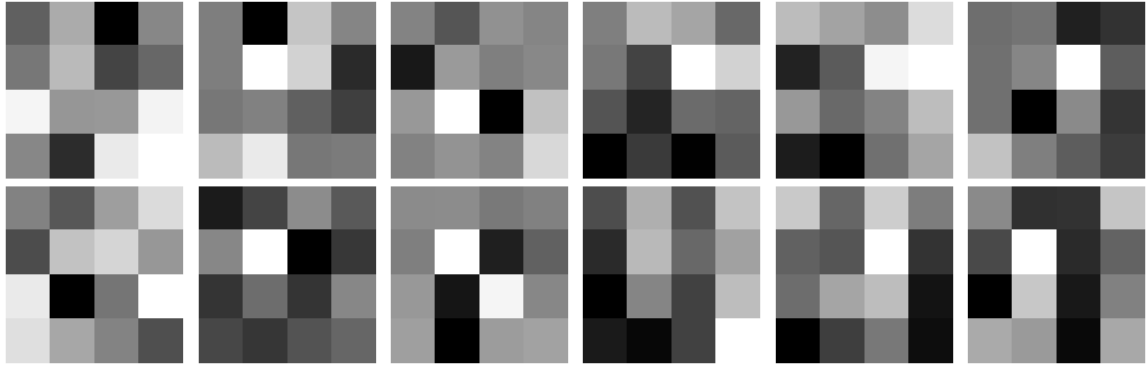


Figure 12: Convolutional Layers: Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

Figure 12: Convolutional Layers: Deeper layers.

The analysis of Case 4 reveals a model that balances generalization with the inherent noise of stochastic updates. As presented in Table 4, the model achieves a Final Test Accuracy of 90.62%, surpassing its training accuracy of 89.81%, which suggests robust generalization capabilities despite the high training duration of 4419.45 seconds. The training dynamics in Figure 10 exhibit visible oscillations in the accuracy curve, a characteristic behavior of online learning where individual pattern updates prevent a perfectly smooth convergence. The Confusion Matrix (Figure 11) identifies persistent topological ambiguities, particularly between digit '4' and '9' (63 misclassifications) and '5' and '3' (45 misclassifications). Visually, Figure 12 demonstrates the network's processing of a handwritten '9', showing the effective preservation of the digit's loop structure in the early convolutional layers (H1, H2) before abstraction into high-level feature maps in the deeper layers (H3, H4).

4.5 Overall Comparison of Online Learning Cases

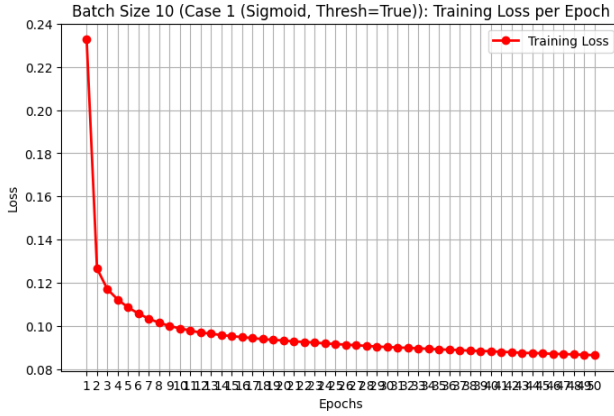
A comparative analysis of the four online learning configurations reveals that the Sigmoid activation function consistently outperforms Softmax in this specific architecture. Cases 1 and 2 (Sigmoid-based) achieved slightly higher test accuracies of 91.67% and 91.21% respectively, compared to Cases 3 and 4 (Softmax-based), which plateaued near 90.6%. Furthermore, the training plots indicate that the Softmax configurations suffered from greater volatility and oscillation during convergence compared to the smoother trajectories of the Sigmoid variants. Although the Softmax activation function is theoretically superior for classification tasks, the observed results may stem from specific hyperparameter combinations, such as the learning rate, the number of epochs, and the use of a linear activation function in the hidden layer. Among all tested scenarios, Case 1 (Sigmoid with Thresholds) proved to be the most robust, delivering the highest stability and the peak test accuracy of 91.67%. Consequently, based on this superior performance, the configuration from Case 1 will be

adopted as the baseline architecture for the subsequent phase of experimentation. In the next stage, Batch Learning algorithm will be experimented, specifically implementing batch sizes of 10 and 100, to investigate the effects of gradient approximation on training efficiency and convergence speed.

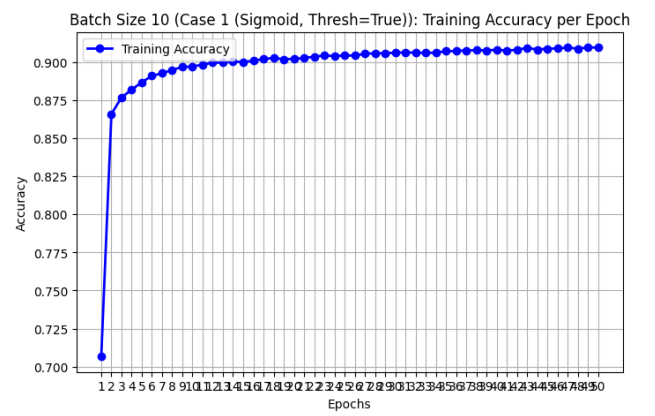
4.6 Analysis of Batch Learning with Case 1 Configuration Batch Size = 10

Metric	Value
<i>Configuration Parameters</i>	
Epochs	50
Activation	Sigmoid
Thresholds	Yes
Batch Size	10
<i>Performance</i>	
Total Training Duration	8236.42 s
Final Training Error	0.0864
Final Training Accuracy	90.97%
Final Test Accuracy	91.70%
Final Test Error	0.0813
Avg. Inference Time	0.9363 ms / pattern

Table 5: Performance Metrics for Case 1 Sigmoid with Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 13: Training performance for Case 1 Batch Size=10 (Sigmoid, Thresh=True) showing Loss (left) and Accuracy (right).

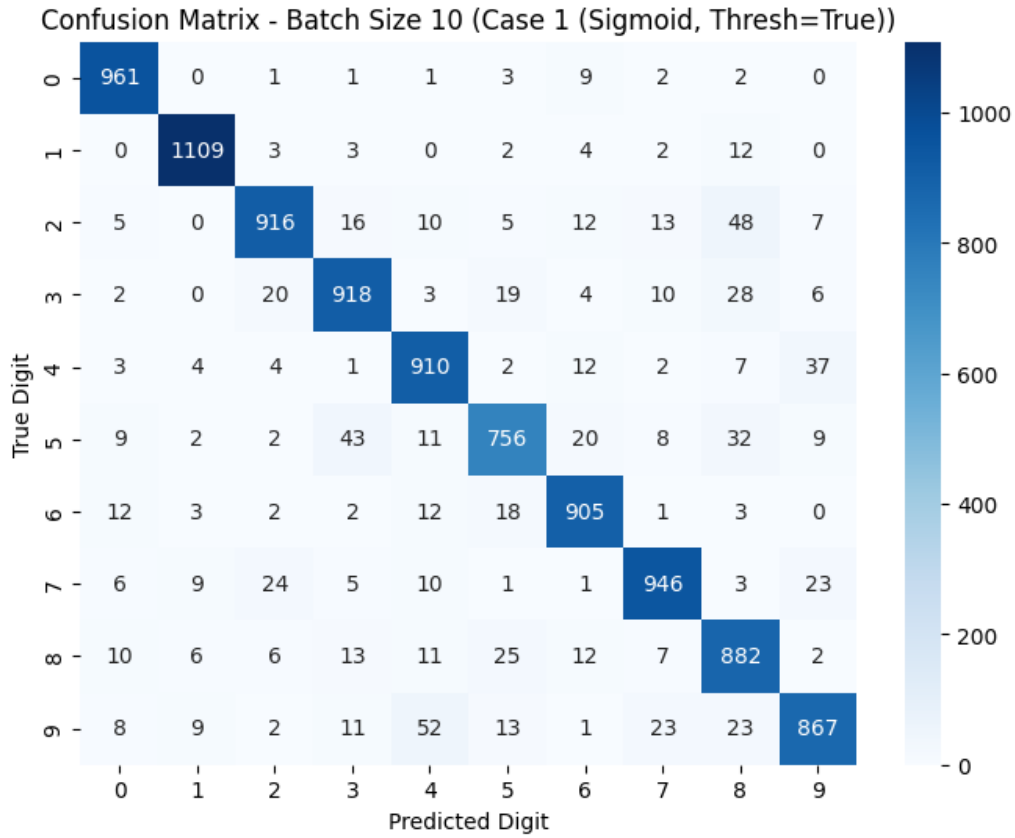


Figure 14: Confusion Matrix for Case 1 (Sigmoid, Thresh=True) on Test Set.

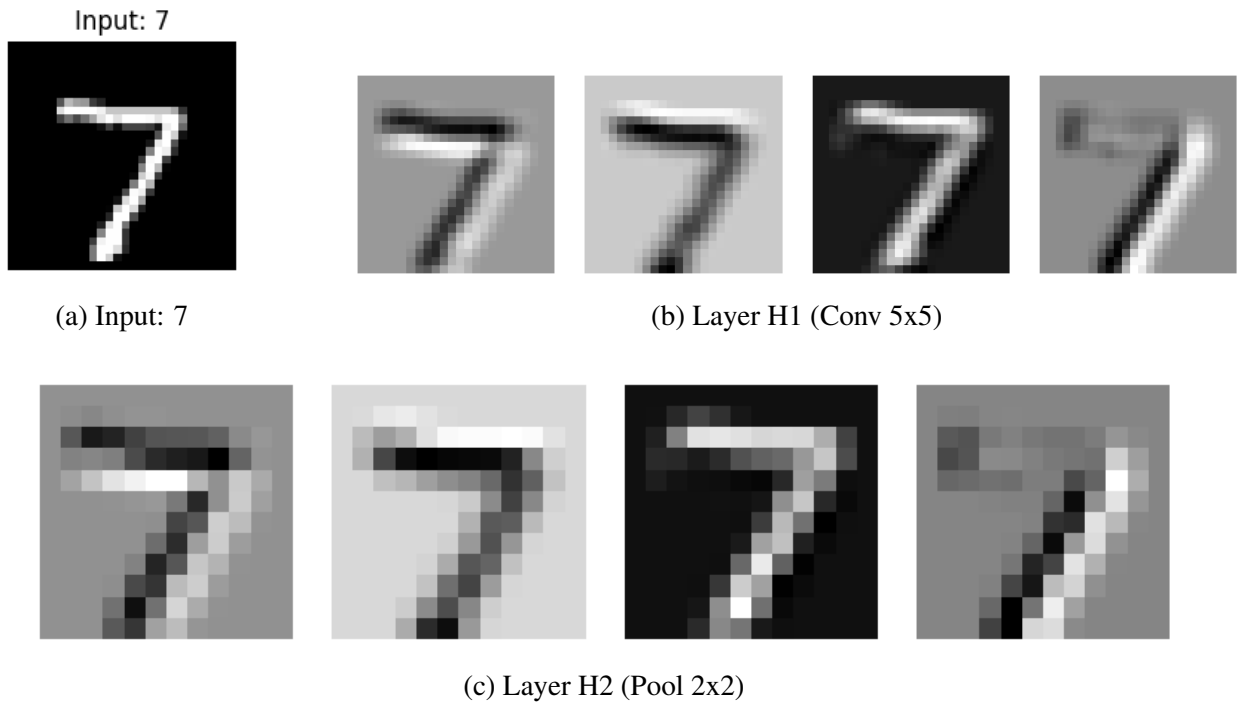
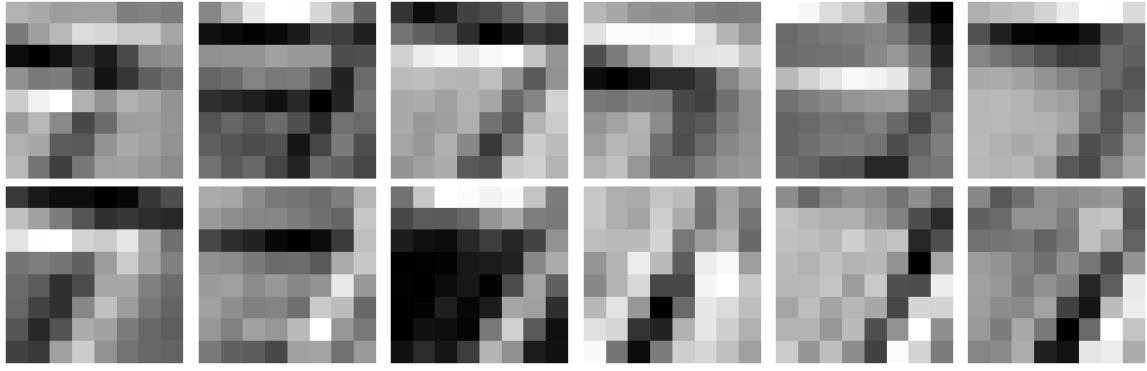
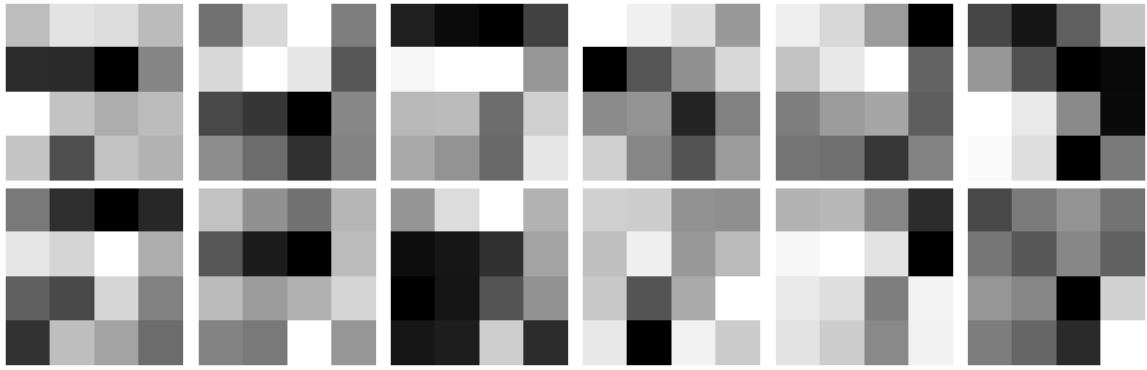


Figure 15: Convolutional Layers: Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

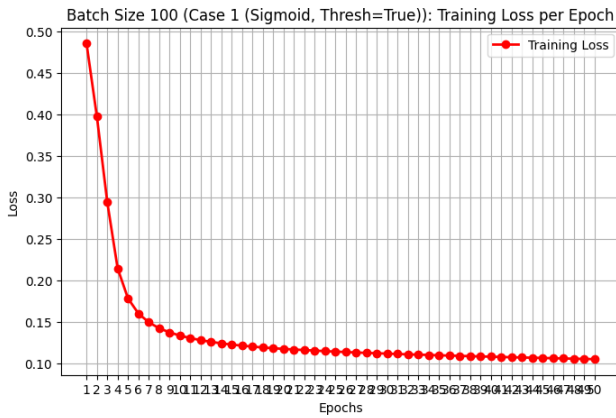
Figure 15: Convolutional Layers: Deeper layers.

The transition to Batch Learning with a batch size of 10 resulted in the highest generalization performance, achieving a Final Test Accuracy of 91.70%. As detailed in Table 5, extending the training to 50 epochs ensured convergence but required a significant computational time of 8236.42 seconds. The training dynamics presented in Figure 13 exhibit remarkably smooth Loss and Accuracy curves, confirming that the mini-batch approach effectively reduces the gradient variance compared to the stochastic noise seen in the online learning experiments. The Confusion Matrix (Figure 14) demonstrates high precision across most classes, though specific morphological confusion persists, notably with 43 instances of digit '5' misclassified as '3' and 37 instances of digit '4' misclassified as '9'.

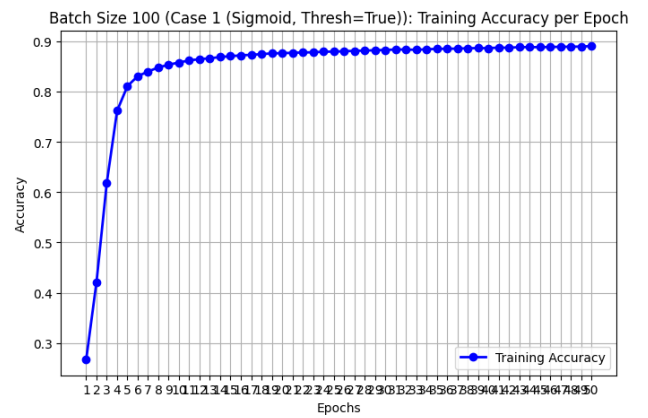
4.7 Analysis of Batch Learning with Case 1 Configuration Batch Size = 100

Metric	Value
<i>Configuration Parameters</i>	
Epochs	50
Activation	Sigmoid
Thresholds	Yes
Batch Size	100
<i>Performance</i>	
Total Training Duration	9371.73 s
Final Training Error	0.1049
Final Training Accuracy	89.12%
Final Test Accuracy	90.22%
Final Test Error	0.0978
Avg. Inference Time	0.9286 ms / pattern

Table 6: Performance Metrics for Case 1 Sigmoid with Thresholds.



(a) Training Loss



(b) Training Accuracy

Figure 16: Training performance for Case 1 Batch Size=100 (Sigmoid, Thresh=True) showing Loss (left) and Accuracy (right).

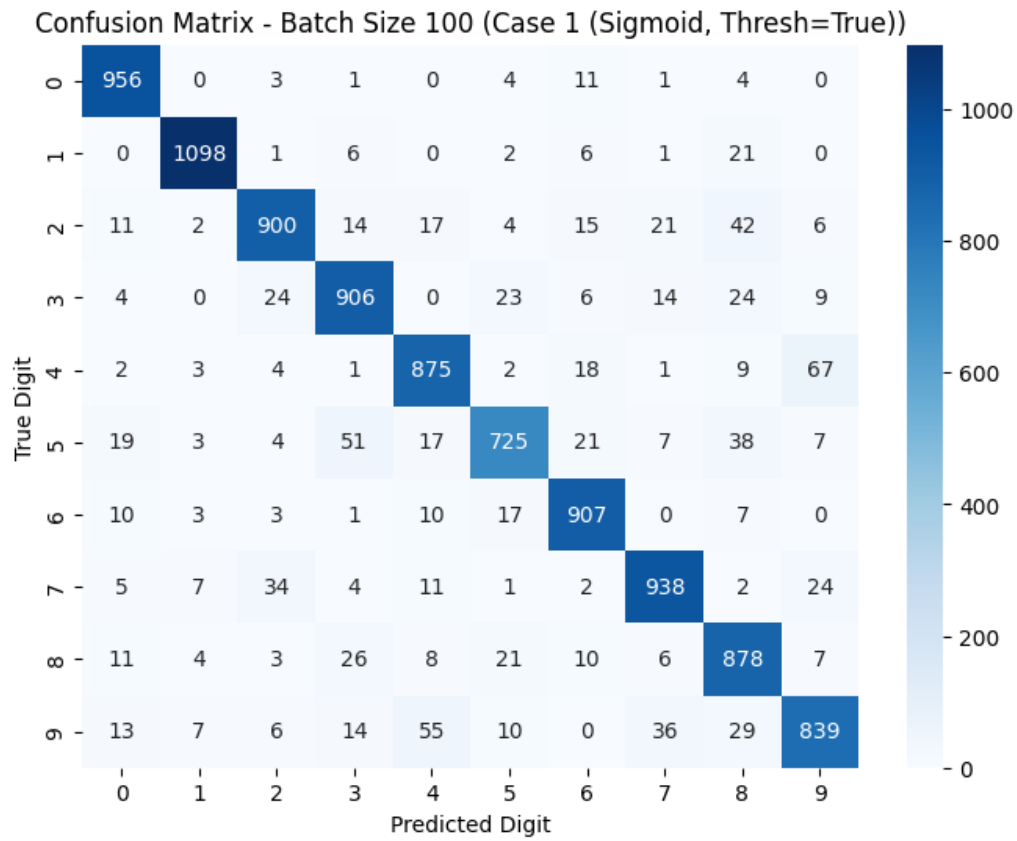


Figure 17: Confusion Matrix for Case 1 (Sigmoid, Thresh=True) on Test Set.

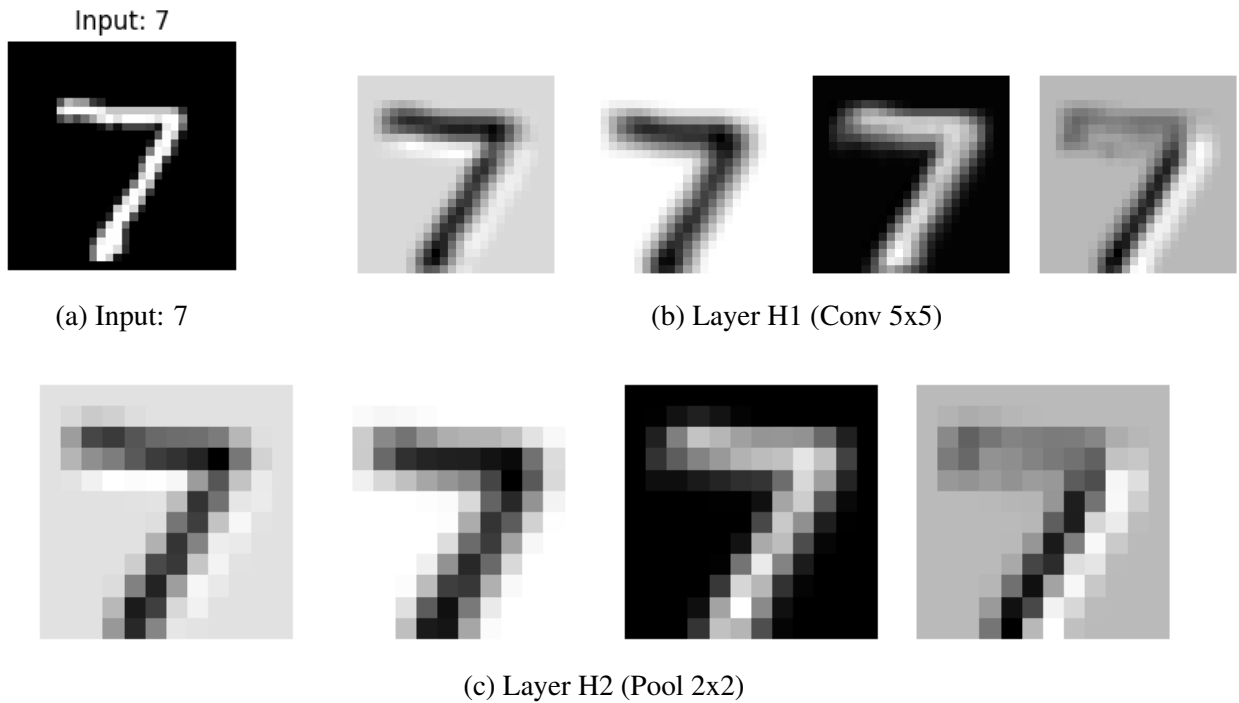
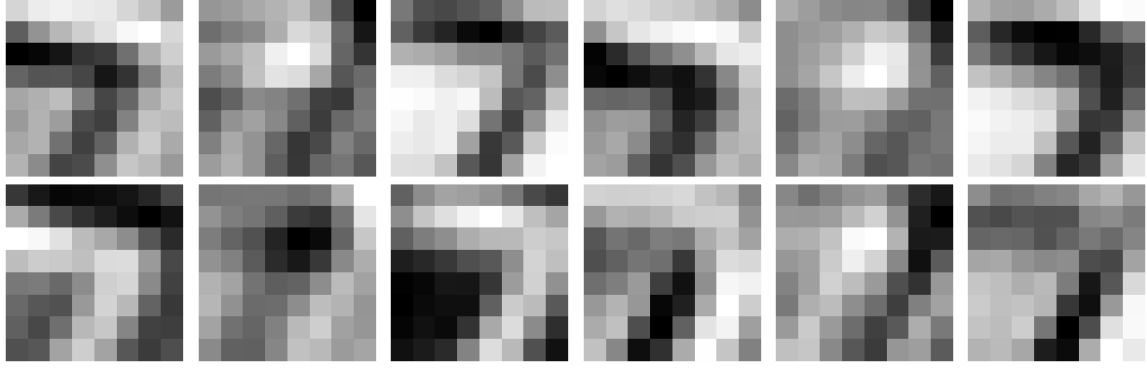
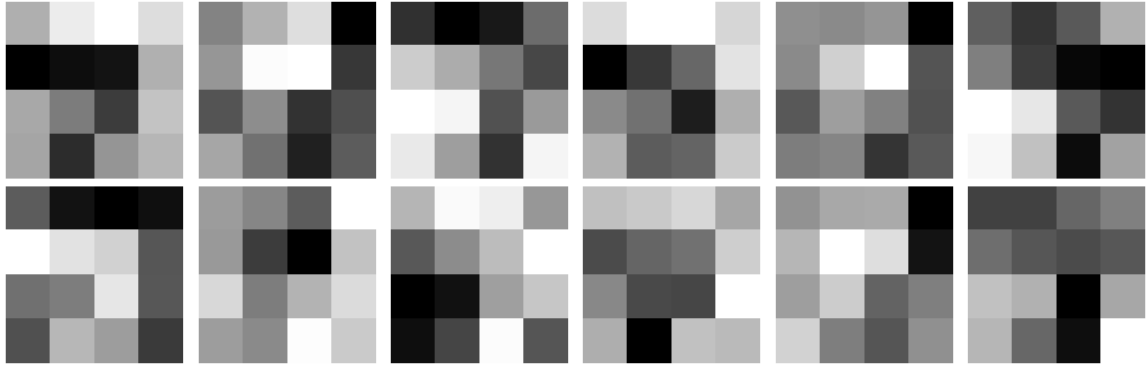


Figure 18: Convolutional Layers: Input and early layers.



(d) Layer H3 (Conv 5x5)



(e) Layer H4 (Pool 2x2)

Figure 18: Convolutional Layers: Deeper layers.

Increasing the batch size to 100 resulted in a noticeable trade-off between gradient stability and final classification performance. As shown in Table 6, while the training process was stable, the Final Test Accuracy dropped to 90.22%, which is lower than both the online learning baseline and the batch size 10 experiment. The training duration also increased to 9371.73 seconds over 50 epochs. Figure 16 displays exceptionally smooth loss and accuracy curves, characteristic of large-batch optimization where gradient noise is minimized; however, this aggressive averaging appears to have limited the model’s ability to escape local minima, resulting in a slightly higher final training error of 0.1049. The Confusion Matrix (Figure 17) reflects this reduction in precision, showing increased misclassification rates for topologically similar digits, specifically 67 instances of ‘4’ predicted as ‘9’ and 51 instances of ‘5’ predicted as ‘3’. Despite the lower accuracy, the Figure 18 confirm that the network successfully learned the necessary structural hierarchies, preserving the distinct edges of the input digit ‘7’ through the convolutional layers.

5 Overall Comments for All Experiments

A comprehensive analysis of the six experimental configurations reveals distinct trade-offs between gradient stability, computational efficiency, and classification accuracy. The Sigmoid activation function consistently outperformed Softmax, demonstrating superior convergence behavior and higher accuracy across both online and batch settings. Among the learning strategies, Batch Size 10 (using Sigmoid with thresholds) emerged as the optimal configuration, achieving the peak final test accuracy of 91.70%. This result indicates in optimization where the mini-batch size provides sufficient gradient approximation to dampen the stochastic noise observed in Online Learning (Case 1, 91.67%), yet retains enough variance to escape local minima capability that was diminished in the Batch Size 100 experiment, which was stuck at 90.22% test accuracy. Ultimately, while online

learning offered a computationally cheaper baseline with high accuracy, the mini-batch approach with $N=10$ provided the best balance of stability and generalization for this specific CNN architecture.

6 Conclusion

This assignment successfully demonstrated the critical impact of hyperparameters such as activation functions selection, thresholding strategies, and batch sizes on the training dynamics and generalization capability of Convolutional Neural Networks. The experimental results conclusively identified the Sigmoid activation function as superior to Softmax for this specific architecture, consistently delivering higher accuracy and smoother convergence profiles. Furthermore, the comparative analysis of learning protocols highlighted the importance of gradient approximation strategies; while online learning provided a robust and high-accuracy baseline, a mini-batch size of 10 proved optimal, achieving the highest overall test accuracy of 91.70% by effectively balancing gradient stability with the stochastic noise requisite for escaping local minima. Conversely, the performance degradation observed at a batch size of 100 (90.22%) illustrates the diminishing returns of excessive gradient averaging. Ultimately, these findings underscore that maximizing neural network performance requires a trade-off between stability and stochasticity during the optimization process.